



Workpath SDK Authorization Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	Authorization Sample on a device.....	4
3	Authorization APIs and Events	5
3.1	APIs.....	5
3.2	Events.....	5
3.3	Broadcast Events.....	5
4	Authorization Sample Flow	6
4.1	Install flow.....	6
4.2	MainActivity Flow and Sign-in event Flow.....	7
5	Key Components for Building an Authorization App.....	8
5.1	Adding Authorization via HPKTool	8
5.2	Setting Permissions in the Android Manifest	8
5.3	Adding a Broadcast Receiver and Registering an Abstract Service	9
5.3.1	Action: AUTHORIZATION_REQUEST	10
5.3.2	Action: AUTHORIZATION_CHANGE_NOTIFICATION	11
5.3.3	Event: authorizeUser()	11
5.4	Enable Authorization with Installation.....	12
5.5	Get Authorization Information	13
5.6	Configuration Handling	14
5.7	ProxyConfiguration	15
5.8	Managing Authorization Settings at Startup	16
5.9	Limitations.....	17
6	Legal notice	18
7	Support.....	19

1 Introduction

AuthorizationSample explains how to use the authorization feature, detailing the API and flow.

The authorization service manages a singleton authorization agent using the Workpath SDK. Through the implementation of the app, it authorizes (grants/denies) access to device functions for authorization front panel users.

This sample application helps developers understand how to use the HP Workpath SDK to implement authorization features effectively.

1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` – Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 Authorization Sample on a device

The AuthorizationSample can retrieve and set the authorization configuration. It also demonstrates how to retrieve the necessary permission list and sign-in methods for the configuration.

4:47 AM

Authorization Sample

VERSION: 1.6.3 (20251111) / 17, API: 1.6.3 / 9

CONFIGURATION

☐ Disable Authorization

Guest

Permission Set

Job Log and Active Jobs

Details or Cancel any user's job

Manage Supplies

Email Settings

Email Setup

Email

Ability to e

☐ Null

Add New Permission To Guest Permission Set

☒ Enable

Enable Sign-in Choice

☒ Enable

Default Sign-in Method

41acce0-a865-4dc5-9e1f-25ab790ebec0

Guest User Overrides

☐ Null

From

emailFromAddress@hp.com

emailFromAddress

To

emailToAddress01 <emailToAddress01@hp.com>

emailToAddress02 <emailToAddress02@hp.com>

X

X

+

Cc

emailCCAddress01 <emailCCAddress01@hp.com>

emailCCAddress02 <emailCCAddress02@hp.com>

X

X

+

Bcc

emailBCCAddress01 <emailBCCAddress01@hp.com>

emailBCCAddress02 <emailBCCAddress02@hp.com>

X

X

+

Subject

emailSubject

Message

Test message

Fax Company Name

HP

Fax Billing Code

123456

GET CONFIGURATION

SET CONFIGURATION

GET PERMISSIONS

GET SIGN-IN METHODS

A main activity of Authorization Sample

3 Authorization APIs and Events

The Workpath SDK provides the following APIs and broadcast events.

3.1 APIs

API	Description
isSupported	Checks if the authorization service is supported on the device.
setConfiguration	Sets the proxy configuration for the authorization service. This method configures the authorization service with a new proxy configuration, which includes settings for guest permissions, sign-in methods, and other parameters. Only one authorization can be set, and if another Workpath authorization app has already set one, it cannot be overwritten.
getConfiguration	Returns the current proxy configuration for the authorization service.
getPermissions	Returns a list of all permissions provided to the device.
getSignInMethod	Returns a list of currently existing sign-in methods.

3.2 Events

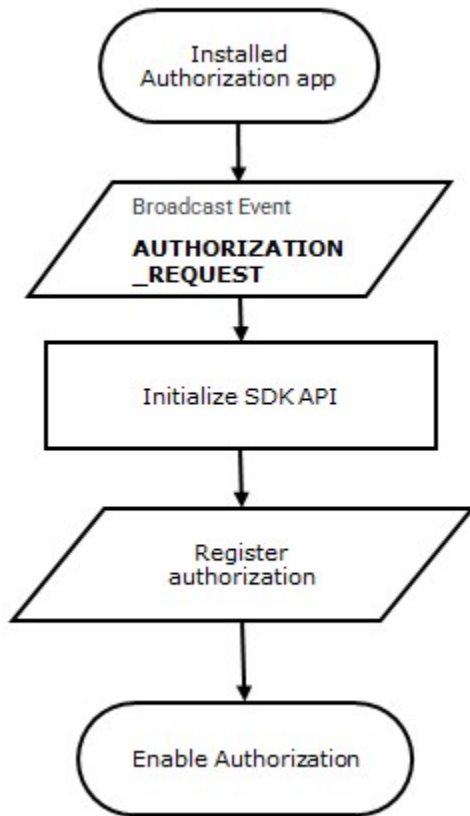
Event	Description
authorizeUser	The process of granting permissions to authenticated users. Currently logged in user information is passed as a parameter, and the user's permission list must be created and returned as a UserAuthorizationResult.

3.3 Broadcast Events

Broadcast	Description
com.hp.workpath.action.AUTHORIZATION_CHANGE_NOTIFICATION	Action sent as a broadcast intent by the Workpath SDK to an app when authorization configuration was changed.
com.hp.workpath.action.AUTHORIZATION_REQUEST	Action sent as a broadcast intent by the Workpath SDK to an app when authorization application is installed for initialize authorization

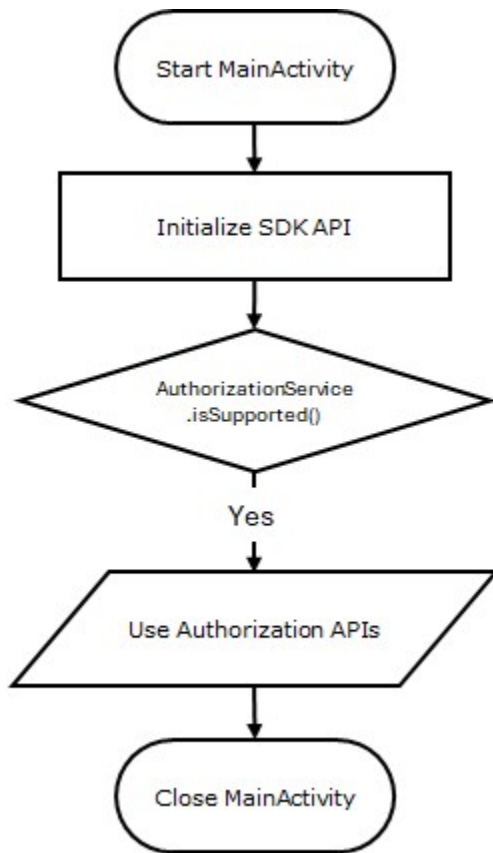
4 Authorization Sample Flow

4.1 Install flow

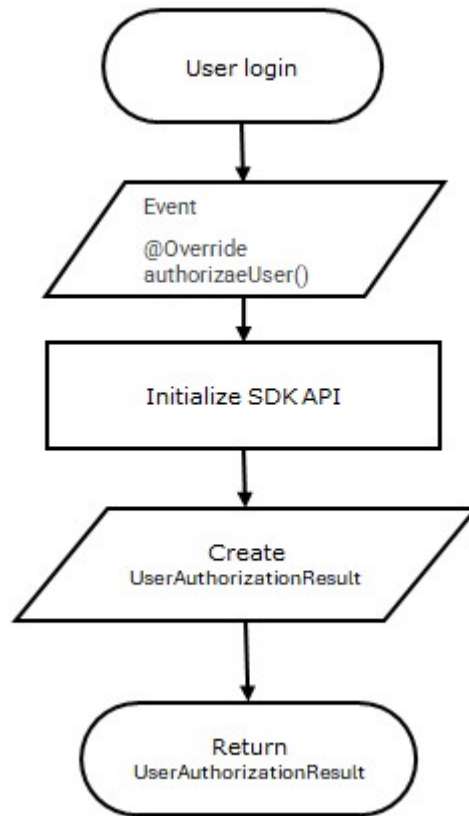


Flow of Initialize authorization

4.2 MainActivity Flow and Sign-in event Flow



Flow of MainActivity



Flow of Sign-in case

5 Key Components for Building an Authorization App

This chapter covers the essential components required to implement the Workpath Authorization App.

To properly receive events, the app must include the following elements.

1. Adding Authorization via HPKTool.
2. Setting Permissions in the Android Manifest.
3. Adding a Broadcast Receiver and Registering an Abstract Service.

5.1 Adding Authorization via HPKTool

Use the HPKTool to activate the authorization feature.

Authorization can be added either by loading an existing HPK file or by creating a new one. You can add localized authorization titles for different languages.

5.2 Setting Permissions in the Android Manifest

To use the authorization feature, the following permission must be added to the AndroidManifest.xml file:

```
1. <uses-permission android:name="com.hp.jetadvantage.link.permission.SERVICES_PERMISSION" />
2. <uses-permission android:name="com.hp.workpath.permission.AUTHORIZATION_PERMISSION" />
```

Declare uses-permission in AndroidManifest.xml

The Workpath service checks for this permission to determine whether the authorization functionality is enabled.

Without this permission, the app will not be able to utilize authorization features.

5.3 Adding a Broadcast Receiver and Registering an Abstract Service

The Workpath service delivers a single event using **two broadcasts** and an **abstract service**.

Actions

- `com.hp.workpath.action.AUTHORIZATION_REQUEST`
- `com.hp.workpath.action.AUTHORIZATION_CHANGE_NOTIFICATION`

Events

- `UserAuthorizationResult authorizeUser(UserAuthorizationData)`

In the sample implementation, each event is handled through the following files:

- **AuthorizationRequestReceiver**
- **ChangeNotificationReceiver**
- **AuthorizeUserService**

```

1. <receiver android:name=".receiver.AuthorizationRequestReceiver"
2.     android:exported="true"
3.     android:permission="com.hp.workpath.permission.AUTHORIZATION_PERMISSION">
4.     <intent-filter>
5.         <action android:name="com.hp.workpath.action.AUTHORIZATION_REQUEST" />
6.     </intent-filter>
7. </receiver>
8.
9. <receiver android:name=".receiver.ChangeNotificationReceiver"
10.    android:exported="true"
11.    android:permission="com.hp.workpath.permission.AUTHORIZATION_PERMISSION">
12.    <intent-filter>
13.        <action android:name="com.hp.workpath.action.AUTHORIZATION_CHANGE_NOTIFICATION"
14.    />
15.    </intent-filter>
16. </receiver>
17. <service android:name=".service.AuthorizeUserService"
18.    android:exported="true"
19.    android:permission="com.hp.jetadvantage.link.permission.SERVICES_PERMISSION">
20.    <intent-filter>
21.        <action android:name="com.hp.workpath.api.action.AUTHORIZATION" />
22.    </intent-filter>
23. </service>

```

Declare service and broadcast receiver in `AndroidManifest.xml`

5.3.1 Action: AUTHORIZATION_REQUEST

The AUTHORIZATION_REQUEST broadcast is triggered when the Workpath Authorization App is successfully installed.

(This flow is also described in section **4.1: Install Flow**.)

Upon receiving this event, the app must call the setConfiguration() API to enable the authorization feature.

This step is essential to fully activate the authorization feature.

```

1. public class AuthorizationRequestReceiver extends BroadcastReceiver {
2.
3.     private static final String ACTION_AUTHORIZATION_REQUEST =
"com.hp.workpath.action.AUTHORIZATION_REQUEST";
4.
5.     @Override
6.     public void onReceive(Context context, Intent intent) {
7.         String action = intent.getAction();
8.         Log.i(MainActivity.TAG, "action: " + action);
9.         if (action != null && action.equals(ACTION_AUTHORIZATION_REQUEST)) {
10.            try {
11.                Workpath.getInstance().initialize(context);
12.                ExecutorService executorService = Executors.newSingleThreadExecutor();
13.                Future<Result> future = executorService.submit(new
SetConfigurationUsingDefaultConfigTask(context));
14.                Result result = future.get();
15.                Log.i(MainActivity.TAG, "AuthorizationRequestReceiver: " +
Logger.build(result));
16.            } catch (Exception e) {
17.                Log.e(MainActivity.TAG, "AuthorizationRequestReceiver: " + e.getMessage());
18.            }
19.        }
20.    }
21. }

```

Implementation in the broadcast receiver (AuthorizationRequestReceiver)

```

1. @Override
2. public Result call() throws Exception {
3.     Result result = new Result();
4.     JSONObject jsonObject = ConfigService.getDefaultConfig(context.get(), result);
5.     if (result.getCode() != Result.RESULT_OK) {
6.         throw new Exception(result.getCause());
7.     }
8.     ProxyConfiguration proxyConfiguration = null;
9.     if (jsonObject != null) {
10.        proxyConfiguration = new Gson().fromJson(jsonObject.toString(),
ProxyConfiguration.class);
11.    }
12.    AuthorizationService.setConfiguration(context.get(), result, proxyConfiguration);
13.    return result;
14. }
15.

```

Implementation in callable task (SetConfigurationUsingDefaultConfigTask)

The overall flow is as follows:

When the `AUTHORIZATION_REQUEST` event is received, the app retrieves the `ProxyConfiguration` data that was previously stored in `DefaultConfig` and then calls the `AuthorizationService.setConfiguration()` API.

Using this API, developers can define the access level for guest users and specify the preferred sign-in method, ensuring that the authorization behavior aligns with the app's security requirements.

Note: In this sample, `DefaultConfig` is used purely for convenience.

This is not a strict guideline; developers are free to implement their own configuration management approach as needed.

5.3.2 Action: `AUTHORIZATION_CHANGE_NOTIFICATION`

The `AUTHORIZATION_CHANGE_NOTIFICATION` broadcast is triggered when a permission is added or when the authorization configuration is changed.

- `ChangeNotificationEventCode.PERMISSION_ADDED`:
Triggered when a new permission is granted.
- `ChangeNotificationEventCode.PROXY_CONFIGURATION_CHANGED`:
Triggered when the proxy configuration has been modified.

These events allow the app to detect and respond to changes in the authorization configuration.

```

1. public class ChangeNotificationReceiver extends BroadcastReceiver {
2.
3.     private static final String ACTION_CHANGE_NOTIFICATION =
"com.hp.workpath.action.AUTHORIZATION_CHANGE_NOTIFICATION";
4.
5.     @Override
6.     public void onReceive(Context context, Intent intent) {
7.         String action = intent.getAction();
8.         if (action != null && action.equals(ACTION_CHANGE_NOTIFICATION)) {
9.             ChangeNotificationEventData data =
intent.getParcelableExtra("changeNotificationEventData");
10.            Log.i(MainActivity.TAG, "ChangeNotificationReceiver: eventCode=" +
data.getEventCode() + ", timestamp=" + data.getTimestamp());
11.
12.            if (data.getEventCode().equals(ChangeNotificationEventCode.PERMISSION_ADDED)) {
13.                Log.i(MainActivity.TAG, "Permissions have changed");
14.            } else if
(data.getEventCode().equals(ChangeNotificationEventCode.PROXY_CONFIGURATION_CHANGED)) {
15.                Log.i(MainActivity.TAG, "Configuration has changed");
16.            }
17.        }
18.    }
19. }
20.

```

Implementation in the broadcast receiver (`ChangeNotificationReceiver`)

5.3.3 Event: `authorizeUser()`

The `authorizeUser()` event is triggered when a user signs in. Through this event, the app grants the appropriate permissions to the user.

```

1. public class AuthorizeUserService extends AbstractAuthorizationService {
2.     @Override
3.     public UserAuthorizationResult authorizeUser(UserAuthorizationData userAuthorizationData) {
4.         Log.i(MainActivity.TAG, "authorizeUser: " + userAuthorizationData.toString());
5.
6.         try {
7.             UserOverrides userOverrides = new UserOverrides.Builder()
8.                 ...
9.                 .build();
10.
11.             Workpath.getInstance().initialize(getApplicationContext());
12.             ExecutorService executorService = Executors.newSingleThreadExecutor();
13.             Future<ArrayList<Permission>> future = executorService.submit(new
GetPermissionsTask(getApplicationContext()));
14.             ArrayList<Permission> permissions = future.get();
15.             Log.i(MainActivity.TAG, "authorizeUser: permissions=" + permissions);
16.
17.             UserAuthorizationResult userAuthorizationResult = new
UserAuthorizationResult.Builder()
18.                 .setAuthorizedUserOverrides(userOverrides)
19.                 .setUserPermissionSet(GetPermissionsTask.getPermissionSetFromPermissions(permissions))
20.                 .build();
21.
22.             return userAuthorizationResult;
23.         } catch (Exception e) {
24.             Log.e(MainActivity.TAG, "authorizeUser: " + e.getMessage());
25.         }
26.         return null;
27.     }
28. }
29.

```

Implementation in the abstract service (AuthorizeUserService)

The sample code demonstrates a scenario where all permissions are granted to the user upon sign-in.

However, in real-world implementation, permissions should be assigned individually based on the user's role or profile.

After determining the appropriate permissions for the user, the app should construct a UserAuthorizationResult object with the relevant data and return it.

This will grant the specified permissions to the user accordingly.

5.4 Enable Authorization with Installation

Workpath Authorization can be enabled automatically upon app installation.

As mentioned in section 5.3.1, once the installation is complete, the app will receive the AUTHORIZATION_REQUEST broadcast.

At that point, the app should apply the initial authorization configuration.

If the AUTHORIZATION_REQUEST broadcast is not received, or if no configuration is applied in response, the current authorization state will remain unchanged.

5.5 Get Authorization Information

The following APIs are available to retrieve the current authorization settings on the device:

- **getPermissions()**

Returns a list of all permissions that have been granted to the device.

```

1. @Override
2. public ArrayList<Permission> call() throws Exception {
3.     Result result = new Result();
4.     ArrayList<Permission> permissions = AuthorizationService.getPermissions(context.get(),
result);
5.     if (result.getCode() != Result.RESULT_OK) {
6.         throw new Exception(result.getCause());
7.     }
8.     return permissions;
9. }
10.

```

Implementation in the callable task (GetPermissionsTask)

- **getSignInMethods()**

Returns a list of all currently available sign-in methods.

```

1. @Override
2. public ArrayList<SignInMethod> call() throws Exception {
3.     Result result = new Result();
4.     if (TextUtils.isEmpty(languageCode)) {
5.         languageCode = context.get().getString(R.string.en_us);
6.     }
7.     ArrayList<SignInMethod> signInMethods =
AuthorizationService.getSignInMethod(context.get(), result, languageCode);
8.     if (result.getCode() != Result.RESULT_OK) {
9.         throw new Exception(result.getCause());
10.    }
11.    return signInMethods;
12. }
13.

```

Implementation in the callable task (GetSignInMethodsTask)

These APIs allow the app to inspect the current authorization state and make decisions accordingly.

5.6 Configuration Handling

You can retrieve or update the current authorization settings on the device using the following APIs:

- **getConfiguration()**

Returns the current proxy configuration used by the authorization service.

```

1. @Override
2. public ProxyConfiguration call() throws Exception {
3.     Result result = new Result();
4.     ProxyConfiguration proxyConfiguration =
AuthorizationService.getConfiguration(context.get(), result);
5.     if (result.getCode() != Result.RESULT_OK) {
6.         throw new Exception(result.getCause());
7.     }
8.     return proxyConfiguration;
9. }
10.

```

Implementation in the callable task (GetConfigurationTask)

- **setConfiguration(ProxyConfiguration or null)**

Updates the authorization configuration.

Note: Passing a **null** value to this API will clear the current authorization settings, effectively disabling authorization.

```

1. @Override
2. public Result call() throws Exception {
3.     Result result = new Result();
4.     AuthorizationService.setConfiguration(context.get(), result, this.proxyConfiguration);
5.     return result;
6. }
7.

```

Implementation in the callable task (SetConfigurationTask)

These APIs are demonstrated in the **AuthorizationSample** provided in the SDK sample project.

By running this sample, you can explore how to use getConfiguration() and setConfiguration() in practice, along with other related authorization APIs.

5.7 ProxyConfiguration

A simple configuration model is used to manage a Proxy's configuration.

The solution constructs a ProxyConfiguration, which includes:

Required:

- **defaultSignInMethod** (String)
: The UUID of the sign-in method to be used as the default for all permissions. This method must be present and enabled on the target device.
- **addNewPermissionToGuestPermissionSet** (Boolean) - Default: false
: When a new permission (i.e., one not listed in the Proxy's GuestPermissionSet) is added, and this flag is set to true, the Proxy will automatically add the new permission to its GuestPermissionSet.

Once permissions are added to the GuestPermissionSet, they persist unless the configuration is changed.

If this flag is set to false, the new permission will be denied to guest users until it is explicitly granted via a ProxyConfiguration update.

- **enableSignInChoice** (Boolean) - Default: false
: Determines whether users are allowed to choose an alternate sign-in method when multiple methods are available and enabled.

Typically configured by an administrator via a management console.

When enableSignInChoice is true, users can select an alternate sign-in method (if available) by pressing the "Advanced" button on the sign-in screen.

Optional:

- **guestPermissionSet** (List<String>)
: A list of permissions that unauthenticated (guest) users are allowed to access, provided those permissions exist on the device. All other permissions will be denied.
- **guestUserOverrides** (UserOverrides)
: A set of override values to be used for guest users (may be null). Not applicable for Whitelisted Remote Apps.
- **permissionToSignInMethodMap** (List<PermissionToSignInMethod>)
: The following is a list of sign-in methods that are currently enabled.

5.8 Managing Authorization Settings at Startup

Authorization settings configured by the application persist across device reboots.

If the application intends to verify or update these settings during device startup, it can implement a `BroadcastReceiver` that listens for the boot completion intent, using the following intent filter.:

```
1. <intent-filter>
2.     <action android:name=" com.hp.jetadvantage.link.action.WORKPATH_READY " />
3. </intent-filter>
```

Once the device receives the `WORKPATH_READY` broadcast, application can apply the desired **Authorization** options using the `setConfiguration()` API.

5.9 Limitations

The following are known limitations when using the Workpath Authorization feature:

- **Only one Workpath Authorization App can be installed** on the device at a time. (It is possible to update the Workpath authorization app if it is already installed)
- If OXPd authorization is enabled, **the Workpath authorization app will not be installed** in this case either.
- If **OXPd authorization overrides** the current configuration, the existing Workpath authorization will be **disabled**.
In this state, the Workpath app will only be able to use the getPermissions() API; all other authorization-related APIs will be unavailable.

When the API cannot be used due to specific conditions or circumstances on the device, the error code 'unavailable' will be returned.

```
Result {  
  "code" : RESULT_FAIL;  
  "errorCode" = ErrorCode.UNAVAILABLE;  
  "cause" = "{error message}";  
}
```

- If you wish to **switch back to Workpath authorization**, you must first **disable the current authorization** and then re-enable it using the setConfiguration() API.

6 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2025 HP Development Company, L.P.

7 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.